The Need for Escape Hatches

Document Number:	D4035R0
Date:	2026-02-25
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: February 2026 (post-Croydon mailing)

1. The Need for Escape Hatches

2. Standard Precedent

3. Established Practice

4. Application Level

5. Structured Concurrency

6. Conclusion

Acknowledgements

References

Abstract

“C++ should make the safe thing easy, and the unsafe thing possible.”

Revision History

R0: February 2026 (post-Croydon mailing)

- Initial version.

---

1. The Need for Escape Hatches

Safe interfaces should be the default. They should validate input, maintain invariants, and protect users from misuse. However, C++ also needs an explicit path for trusted data - when the precondition is already satisfied at a boundary and re-validation is pure overhead.

This pattern appears in the standard library, in production Boost libraries, in the current `cstring_view` proposal, and in coroutine-based concurrency libraries. Four independent examples follow.

2. Standard Precedent

The standard library provides `std::condition_variable` [1], which requires `std::unique_lock<std::mutex>`. This constraint enables optimizations. When the constraint is too narrow - when the user holds a different lock type - `std::condition_variable_any` [1] provides the explicit broader path:

```
std::mutex mtx;
std::shared_mutex smtx;

// Safe default: constrained to unique_lock<mutex>.
std::condition_variable cv;
std::unique_lock<std::mutex> lk(mtx);
cv.wait(lk);

// Escape hatch: works with any lockable.
std::condition_variable_any cv_any;
std::shared_lock<std::shared_mutex> slk(smtx);
cv_any.wait(slk);
```

The constrained interface is the default. The broader interface is explicit and named. Both exist in the standard.

3. Established Practice

Boost.URL[2] provides `pct_string_view`, a non-owning reference to a valid percent-encoded string. Construction from untrusted input validates the encoding and throws on failure:

```
// Safe default: validates percent-encoding.
pct_string_view s("Program%20Files");    // OK
pct_string_view bad("100%pure");         // throws
```

Internally, the library's own parser has already validated the encoding before extracting URL components. Repeating that validation would be redundant. A separate function bypasses it at the trusted boundary:

```
// Escape hatch: precondition already satisfied by the parser.
pct_string_view s = make_pct_string_view_unsafe(data, size, decoded_size);
```

This pattern was adopted independently by three Boost libraries: Boost.URL (`make_pct_string_view_unsafe` [3]), Boost.Process (`basic_cstring_ref` [4]), and Boost.SQLite (`cstring_ref` [5]). Three independent libraries arriving at the same design is not coincidence. It is convergence on a structural need.

4. Application Level

On BSD-derived systems, directory iteration exposes a filename pointer and a filename length via `dirent` (`d_name` and `d_namlen`) FreeBSD `readdir(3)` [6] and FreeBSD `dirent.h` [7]. POSIX requires that path components are null-terminated and contain no embedded null bytes POSIX Base Definitions[8]. Rescanning each name in a validating constructor repeats work the operating system already did.

```
void visit_directory(DIR* dir)
{
    while (dirent* de = ::readdir(dir))
    {
        const char* p = de->d_name;
        std::size_t n = de->d_namlen;

        // Trusted boundary: OS already satisfied the precondition.
        cstring_view name = cstring_view::unsafe(p, n);
        consume(name);
    }
}
```

The safe path remains the default for untrusted input. The unsafe path exists for proven preconditions and zero additional runtime cost.

5. Structured Concurrency

Structured concurrency enforces the same discipline for asynchronous lifetimes that structured control flow enforces for execution order: activations nest, scopes nest, a parent outlives its children, and RAII works. Capy[9] provides `run` as the structured default and `run_async` as the explicit escape hatch:

```
recycling_frame_allocator alloc;

// Safe default: structured, caller suspends until child completes.
co_await run(ex, alloc)(my_coro());

// Escape hatch: fire-and-forget, caller continues immediately.
run_async(ex, alloc,
    []() { /* Result handler. */ },
    [](std::exception_ptr) { /* Error handler. */ }
)(my_coro());
```

The structured path is the default. The unstructured path requires a longer name and explicit handlers - the caller must decide up front how results and errors are delivered. The escape hatch still provides guardrails: a work guard keeps the execution context alive, a stop token enables cooperative cancellation, and typed handlers prevent silent result loss.

The consequences of misuse are higher in concurrency than in data validation - a wrong `pct_string_view` produces garbage data, while a wrong detached task produces data races. That severity is precisely why the escape hatch must be designed rather than left to raw `std::thread` or `std::async`, which offer none of these guardrails. Eliminating the escape hatch does not eliminate unstructured concurrency; it pushes programmers toward worse tools.

6. Conclusion

The standard library already provides constrained defaults with explicit broader counterparts. Production libraries independently converge on the same pattern for trusted boundaries. The same principle governs concurrency structure. This paper asks for no wording and no poll. It asks the committee to recognize a design value: safe by default, with explicit escape hatches where zero-cost composition requires them.

How explicit escape hatches interact with Hardening, Contracts, and Erroneous Behavior is a related question that deserves separate treatment.

Acknowledgements

The author thanks Jonathan Wakely, Jan Schultke, Pablo Halpern, and Nevin Liber for discussion and feedback on safe and unsafe construction paths.

References

[1] C++ Working Draft, `condition_variable` https://eel.is/c++draft/thread.condition.condvar, `condition_variable_any` https://eel.is/c++draft/thread.condition.condvarany

[2] Boost.URL, https://github.com/boostorg/url

[3] Boost.URL, `pct_string_view.hpp`, https://github.com/boostorg/url/blob/develop/include/boost/url/pct_string_view.hpp

[4] Boost.Process, `cstring_ref.hpp`, https://github.com/boostorg/process/blob/develop/include/boost/process/v2/cstring_ref.hpp

[5] Boost.SQLite, `cstring_ref.hpp`, https://github.com/klemens-morgenstern/sqlite/blob/develop/include/boost/sqlite/cstring_ref.hpp

[6] FreeBSD `readdir(3)`, https://man.freebsd.org/cgi/man.cgi?query=readdir&sektion=3

[7] FreeBSD source, `sys/sys/dirent.h`, https://cgit.freebsd.org/src/tree/sys/sys/dirent.h

[8] The Open Group Base Specifications Issue 8, https://pubs.opengroup.org/onlinepubs/9799919799/

[9] Capy, https://github.com/cppalliance/capy